From Model Swap to Workflow Engineering: Upgrading FinRobot for Equity Research

Y Yang Yu 8 min read · Just now

When I first read the assignment, I thought the main task would be simple: replace FinRobot's original base model with stronger frontier LLMs and compare the generated equity research reports. After running the experiments, I realized that the harder problem was not only model quality. The more important question was:

What kind of workflow makes an LLM useful inside an equity research pipeline?

Equity research generation is not a one-shot prompting task. A reliable system needs stable financial data access, reproducible batch runs, consistent evaluation, and a way to preserve report-level coherence across different sections. Because of that, this project became less about simple model benchmarking and more about workflow engineering around frontier models.

In this project, I upgraded FinRobot with a reproducible multi-model experiment framework, financial data caching, two-stage evaluation, section-level scoring, and a role-mixed analyst/critic workflow. The main lesson was clear: model quality matters, but model orchestration, evaluation design, and role assignment matter just as much.

Project Goal

The assignment required two major components.

The first was base model replacement: replacing the original FinRobot model setup with stronger frontier LLMs and comparing the results.

The second was equity research workflow enhancement: adding a meaningful improvement inspired by Claude-style analyst reasoning, such as stronger investment thesis generation, catalyst tracking, risk analysis, critique, or report structure.

I designed the project around two questions:

- How do OpenAI, Claude, and Gemini perform as base models for FinRobot equity research reports?
- Can workflow-level improvements produce better reports than simply choosing the best single model?

The required companies were included in every major experiment:

- NVDA — NVIDIA
- AMD — Advanced Micro Devices
- INTC — Intel
- AAPL — Apple
- GOOGL — Alphabet
- These companies created a useful test universe because they include semiconductor firms, consumer technology, and internet/AI exposure.

What I Upgraded

Batch Model Replacement

The first upgrade was moving away from manual single-run testing.I added a reproducible batch runner, `run_experiments.py`, which runs model-by-ticker combinations and stores all outputs under a consistent run folder. This made it possible to compare reports across companies and model settings instead of relying on isolated examples.Each run produced structured artifacts such as `reports_index.json`, `evaluation_summary.json`, per-task logs, manifests, and final scoreboards.This changed the project from an ad-hoc test into a repeatable experiment.

FMP-Aware Caching and Fallback

One of the biggest practical bottlenecks was not the LLM itself. It was the financial data layer.FinRobot depends on financial market data, and the FMP free-tier API can quickly become a constraint when running many model and ticker combinations. Without caching, repeated experiments would create unnecessary API calls and unstable failures.To solve this, I added endpoint-level caching in `market_data_api.py` .The cache tracks three possible data origins:

- `fresh_fetch` : the endpoint was successfully fetched from the API.
- `cache_hit` : the same endpoint, ticker, and parameters were already fetched on the same local date.
- `fallback_stale_cache` : the network fetch failed, so the pipeline used an existing cached payload.

This made the experiment loop more stable and reduced repeated API calls during same-day reruns.This was an important engineering lesson: in real equity research automation, data reliability can become a bottleneck before model quality does.

Two-Stage Evaluation

Manual review is expensive, inconsistent, and hard to scale. To compare many reports, I implemented a two-stage evaluation process.The first stage uses a lightweight judge model, `gpt-4o-mini` , to score every report independently.The second stage uses a stronger judge model, `gpt-5-nano` , only when candidates are close enough to require pairwise review.This design gave a better balance between cost and evaluation quality. Instead of using a high-tier judge for every report, the pipeline only escalates close cases.The evaluation setup was:

Stage	Judge Model	Purpose
Stage 1	`gpt-4o-mini`	Fast independent scoring
Stage 2	`gpt-5-nano`	Pairwise review for close candidates

This allowed me to compare reports more consistently across OpenAI, Claude, Gemini, section-mixed reports, and role-mixed reports.

Section-Level Model Selection

After generating baseline reports, I tested a natural idea:

What if each model is best at different report sections?

For example, one model might write a stronger investment overview, while another might write better risk analysis or final takeaways. To test this, I added `evaluate_sections.py`, which scores individual report sections and creates a global section mapping.Then `build_mixed_reports.py` uses the best-scoring model for each section and rebuilds a theoretically stronger mixed report.In theory, this should produce a better report.In practice, it did not.The section-mixed strategy became one of the most important negative findings of the project.

Claude-Inspired Enhancement: Role-Mixed Analyst/Critic Workflow

After section mixing failed, I tested a different strategy: instead of mixing sections mechanically, I assigned different models to different roles.This became my main Claude-inspired equity research enhancement. Instead of treating Claude only as another base model, I used it as a critic/reviewer agent to revise thesis-bearing sections, including investment_overview, risks, and major_takeaways. The critic stage was designed to check whether the investment thesis was supported by financial evidence, whether the risk section directly challenged the thesis, and whether the final takeaway was consistent with the valuation logic.

I implemented this in `run_role_mixed.py`.The two role-mixed settings were:

Setup	Analyst	Critic
`role_mixed_a`	Claude	Gemini
`role_mixed_b`	Gemini	Claude

The critic was not asked to rewrite the entire report. Instead, it revised the most thesis-bearing sections:

- `investment_overview`
- `risks`
- `major_takeaways`

This design was inspired by an analyst/reviewer workflow. The analyst produces the initial research draft, and the critic checks the logic, identifies weak reasoning, improves risk framing, and sharpens the final thesis.This

was closer to how an equity research report is actually reviewed.Experiment Setup

The accepted benchmark run was:

```
run_id = 20260501_005147
```

The tested base models were:

- `openai_gpt-5.4`
- `claude_claude-opus-4-6`
- `gemini_gemini-3.1-pro-preview`

The experiment covered all five required companies: NVDA, AMD, INTC, AAPL, and GOOGL.

The pipeline compared three types of setups: unified model baselines, section-mixed reports, and role-mixed analyst/critic reports.

The main evaluation output was stored in `final_scoreboard.csv`.

Results

The final mean scores were:

Setup	Mean Final Score
`openai_gpt-5.4`	49.192
`claude_claude-opus-4-6`	53.464
`gemini_gemini-3.1-pro-preview`	53.732
`mixed`	47.132
`role_mixed_a` — Claude analyst + Gemini critic	47.948
`role_mixed_b` — Gemini analyst + Claude critic	54.828

The best overall setup was:

```
role_mixed_b = Gemini analyst + Claude critic
```

It achieved the highest mean final score: **54.828**.

It also won all five ticker-level role-mixed comparisons.

Baseline Model Winners

Across the baseline reports, the best model varied by ticker:

Ticker	Baseline Winner
AAPL	Gemini
AMD	Gemini
GOOGL	Claude
INTC	Claude
NVDA	OpenAI

This result suggests that no single model dominated every company. Different companies and report contexts rewarded different model strengths.Gemini performed well for AAPL and AMD, Claude performed better for GOOGL and INTC, and OpenAI produced the strongest baseline for NVDA.However, the best overall result did not come from choosing one model for all tasks. It came from assigning models to complementary roles.

Why Section Mixing Failed?

The section-mixed experiment tested a tempting but flawed assumption:If we take the best section from each model, the final report should be better.The results showed the opposite.The mixed report achieved a mean final score of **47.132**, lower than the major baseline models.The likely reason is that an equity research report is not just a collection of independent sections. The investment overview, financial discussion, risk section, and final takeaways need to share the same thesis.When sections from different

models are stitched together, the report may lose thesis continuity, consistent risk framing, smooth transitions, a unified conclusion, and coherent valuation logic.

This was the key lesson:Local section quality does not automatically create global report quality. A model can write an excellent risk section in isolation, but if that risk section does not match the investment thesis or conclusion, the full report becomes weaker.

Why Role Mixing Worked Better

The role-mixed strategy performed better because it preserved report coherence.Instead of selecting isolated sections from different models, the pipeline first let one model generate a complete analyst draft. Then a critic model revised only the most important thesis-bearing sections.This kept the report's structure intact while still allowing another model to improve the logic.The best configuration was:

```
Gemini analyst + Claude critic
```

This result suggests that Gemini was effective at generating the initial analyst-style report, while Claude was especially useful for critique and revision.Claude's critic role improved sections where reasoning quality matters most: investment thesis, risk analysis, and major takeaways.This supports the idea that the best multi-agent system may not be the one that uses the "best" model everywhere. It may be the one that assigns each model to the role where it adds the most value.

What Worked

The strongest improvements came from three areas.First, FMP-aware caching made the pipeline more reliable under API limits. Without it, long experiment loops would have been difficult to reproduce.Second, the two-stage judge system made evaluation more scalable. It avoided using expensive review on every output while still allowing close cases to receive deeper comparison.Third, the analyst/critic setup produced the strongest final result. It improved report quality without destroying coherence.The most important finding was that targeted critique worked better than naive section recombination.

What Did Not Work

The main failed experiment was section mixing.Although the section-level approach made sense in theory, it produced weaker full reports. The problem was not that the selected sections were individually bad. The

problem was that the combined report lacked a consistent narrative.This is especially important in equity research because a good report needs to move logically from company context to financial evidence, then to investment thesis, risk analysis, and final recommendation.If these parts are written with different assumptions, the report becomes less useful even if each section looks strong on its own.

Limitations

This experiment has several limitations.First, the results are conditioned on the specific prompts, token budgets, and judge setup used in this run. A different scoring rubric or threshold could change the exact ranking.Second, token usage across model providers is not fully comparable. OpenAI, Claude, and Gemini may handle long context and output generation differently.Third, the results should not be treated as a universal claim that Gemini is always the best analyst or Claude is always the best critic. The conclusion is specific to this accepted run, this pipeline, and these five companies.Fourth, the evaluation itself depends on LLM judges. While two-stage evaluation improves consistency, it does not fully replace human financial judgment.

Reproducibility

The accepted run can be reconstructed from:

```
output/storage/20260501_005147/
```

Key files include:

- `reports_index.json`
- `evaluation_summary.json`
- `global_section_mapping.json`
- `evaluation_mixed_summary.json`
- `role_mixed_manifest.json`
- `evaluation_role_mixed_summary.json`
- `final_scoreboard.csv`

The main scripts are:

- `run_experiments.py`
- `evaluate_sections.py`

- `build_mixed_reports.py`
- `run_role_mixed.py`

Together, these files and scripts make the experiment reproducible and allow the final claims to be checked against generated artifacts.

Final Takeaway

The best result in this project did not come from simply selecting the strongest standalone model.

It came from building a better workflow around the models.

For the accepted run, the strongest configuration was:

```
Gemini analyst + Claude critic
```

This setup preserved report coherence better than section mixing and achieved the highest mean final score.The broader takeaway is that equity research automation is a workflow problem as much as a model problem. Strong LLMs are necessary, but not sufficient. A reliable system also needs data-layer resilience, reproducible orchestration, scalable evaluation, and role-aware revision.

In other words:

Better models improve the pipeline, but better workflow design makes the models useful., but better workflow design makes the models useful.

Written by Yang Yu

0 followers · 1 following

Edit profile

No responses yet

Yang Yu

What are your thoughts?